

KV Cache Optimization in LLMs: A Survey, Benchmark, and Decision Framework for Efficient Inference

1st Aayush Chhuka

*Department of Computer Science and Engineering
National College of Engineering
Lalitpur, Nepal
aayushchhuka20@gmail.com*

2nd Aditya Brajacharya

*Department of Computer Science and Engineering
National College of Engineering
Lalitpur, Nepal
aadityabrajacharya123@gmail.com*

3rd Sudip Dhungana

*Department of Computer Science and Engineering
National College of Engineering
Lalitpur, Nepal
sudipdhungana02@gmail.com*

Abstract—Large Language Models rely on the Key-Value (KV) cache to save on computing projections of previously generated tokens in autoregressive inference. Nevertheless, the KV cache scales linearly with the context size and becomes a major limiting factor for GPU memory, memory bandwidth, and inference latency. In this work, we propose a taxonomy based on bottleneck identification and analyze ten KV cache optimizations techniques belonging to four different classes attention level redesign, quantization, compression, and memory management in a unified way with respect to a set of common metrics. The analysis is based on a reproducible benchmarking framework, *kvbench*, measuring the peak GPU memory, CPU resident-set size, wall-clock latency, throughput, and analytical KV cache size on an identical set of prompts, seeds, and (context, batch) settings. Three optimizations whose kernels are proprietary (NVFP4, MiniKV, xKV) are simulated by high-fidelity analytical models. The proposed framework and the comparison serve as a decision framework for choosing KV cache optimization techniques in various scenarios of memory, latency, and accuracy constraints.

Index Terms—KV cache, large language models, inference optimization, quantization, compression, memory management, attention, benchmarking.

I. INTRODUCTION

Recently, Large Language Models (LLMs) have become a core foundation in AI-driven applications due to the sole reason of powerful understanding and generation of natural language. Unlike traditional AI systems, LLMs provide great flexibility and handle wide range of tasks with enhanced adaptability [1]. To manage active information, LLMs utilize a short-term memory technique known as the context window. Within this window, the model processes previous conversational turns, external documents, and its own prior outputs. To generate logical text, LLMs rely on an autoregressive process which generates tokens one by one based on all previous tokens to understand the broader context of the sequence [2].

A main challenge for LLM deployment is how models manage, access and retain information across interactions. Due to this autoregressive behaviour, the computing cost of a model increases as it recalculates the mathematical relation of previous tokens each and every time a new token is generated [2]. This leads to computational overhead within the context window. In some cases input context can be tens and hundreds of times longer than the output it generates [3].

To overcome the limitation of recomputation of past token, the key-value (KV) cache is used which stores the intermediate attention states (Keys and Values) of past tokens in memory during the processing phase [4]. So when new tokens are generated it does not recalculates but reuse the prior computation. Resulting in reduction of latency and make autoregressive generation more efficient.

Although the KV cache has improved the inference efficiency of Large Language Models (LLMs), its memory footprint grows linearly with context length which increased memory consumption, introduce bottlenecks in GPU memory capacity, memory bandwidth, inference throughput, and latency, especially for long-context and multi-user inference [4].

As LLMs can handle longer context, different methods has been introduced to improve and optimize performance of KV cache and most of them focuses on single technique. Limited work has been published which organize these methods, making it difficult to understand their concepts, how they work, their advantages and their limitation. This report organize the KV cache and the different optimization methods used in modern LLMs and covers techniques such as KV cache compression, quantization, token selection, offloading, and memory management. These techniques are then compared based on memory usage, computational cost, inference speed and their applications and provide appropriate guideline for selecting suitable methods under different memory, latency,

and computational constraints.

II. LITERATURE REVIEW

A. Background

KV cache is crucial for LLM deployment as it reduces latency by avoiding recomputing past tokens, lowering operational costs with less compute requirements, and enables deployment on resource-constrained devices by managing large memory and optimizing data transfers. KV Cache stores intermediate key which represents identity of token and value vectors which represents actual content of that token from previous tokens to avoid redundant computation during generation [4]. And when new token is generated, the key and value pair are appended to the cache. The KV cache size is expressed as:

$$KV_{\text{per token}} = 2 \times H \times L \times B \times D \quad (1)$$

In Equation (1), [4] H is the number of attention heads; D the dimension of each head; L the number of transformer layers; B the bytes per element. The factor 2 accounts for storing both the key tensor and the value tensor. These parameters are fixed for a given model, the KV per token is constant for a model.

$$KV_{\text{cache size}} = KV_{\text{per token}} \times \text{Context Length} \quad (2)$$

Equation (2) shows that the total KV-cache memory increases linearly with the context length. As modern LLMs support context windows containing tens or hundreds of thousands of tokens, the KV cache becomes a major bottleneck in terms of GPU memory consumption, memory bandwidth, and inference latency [2], [4].

For ease of reference, the symbols used throughout the paper are collected below:

- H — number of attention heads.
- D — dimension of each attention head.
- L — number of transformer layers.
- B — bytes per element (e.g. $B = 2$ for FP16, $B = 1$ for INT8/FP8, $B = 0.5$ for FP4).
- N — total number of tokens currently in the cache (context length).
- g — GQA group size; g query heads share a single KV head.
- r — rank of the low-rank projection used by low-rank compression ($r < D$).
- k — number of consecutive transformer layers that share a single xKV representation.

Each optimization techniques reduces either one of the terms of Equation (1) to optimize and improve efficiency of KV cache. To deal with this bottleneck, there are different techniques which can be categorized into attention-level redesign, quantization, compression, and memory management.

B. Attention Level Techniques

Ainslie et al. [5] introduces Grouped-Query Attention (GQA), which groups multiple query heads to share same key and value, as shown in Fig. 1 reducing cache size while losing little output quality. GQA stays in between two attention techniques: standard Multi-Head Attention (MHA), where each query head has its own key and value, and Multi-Query Attention (MQA), where all query heads share a single key and value.

The result from the experiment shows that GQA inference speed is close to that of Multi-Query Attention (MQA) and maintains performance similar to that of standard Multi-Head Attention. The authors also introduce an effective way to upgrade existing MHA models into GQA models using mean pooling, requiring only a small amount of time and cost to adapt [5].

C. KV Cache Quantization

KV Cache Quantization is the optimization technique which is used to reduce memory requirement to store key and value tensors. Normally, these tensors are stored in high precision format usually FP16, instead of that these cached tensors are represented in lower precision format such as FP8, FP4. This helps in decreasing memory footprint and allows longer context length while maintaining model accuracy [2].

To reduce memory usage, multiple studies have introduced different KV cache quantization methods. FP8 KV cache is widely used because of its balance in memory savings and in model accuracy. A newer method, NVFP4 KV cache has been introduced by NVIDIA, which stores key and value tensors in 4-bit floating point format, reducing KV cache memory by half compared to that of FP8, allowing longer context lengths, supports more users, and lowers memory bandwidth during decoding. Cached values are dequantized for attention computation and quantized again before being stored [2].

Another method is MixQuant [3] which uses mixed precision quantization by providing different precision levels to different cached tensors based on their importance. This reduce memory while maintaining better accuracy than using low level precision for all tensors.

MiniKV [6] improves efficiency of KV cache by combining 2-bit quantization and removes less important cached tokens. It not only reduce size of KV cache through quantization, it also frees memory by keeping only the most useful information. MiniKV reduces KV cache memory by more than 80% while maintaining accuracy and providing faster inference than methods that use only quantization.

Table I summarizes recent KV cache quantization techniques based on the findings in [2], [3], [6].

Although KV Cache Quantization reduces memory usage, it reduces generation quality for a longer context because of using low precision tensor format. As a result, current research focuses on finding the right balance between memory efficiency and model accuracy to enable the practical deployment of modern LLMs [2], [3], [6].

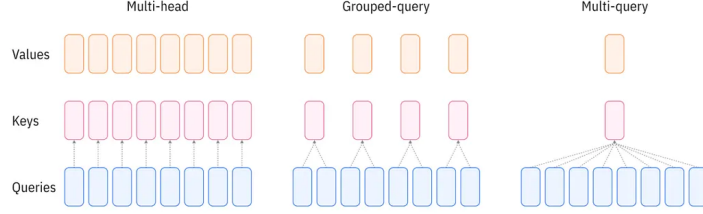


Fig. 1. An structural overview of the Grouped-Query Attention (GQA) mechanism compared alongside Multi-Head Attention (MHA) and Multi-Query Attention (MQA) [5].

TABLE I
COMPARISON OF KV CACHE QUANTIZATION TECHNIQUES.

Method	Precision	Main Benefit	Trade-off
FP16	16-bit	Highest accuracy	Large memory usage
FP8	8-bit	Lower memory	Slight accuracy loss
NVFP4	4-bit	Smaller cache	Quantization overhead
MixQuant	Mixed precision	Better accuracy-memory trade-off	Higher complexity
MiniKV	2-bit + adaptive eviction	80% compression	Custom kernels

D. KV Cache Compression

When context length increases, the KV cache occupies a large portion of GPU memory. While quantization reduces the precision of cached tensors, KV-cache compression decreases the amount of information stored within the key-value representations by removing redundancy. It reduces memory consumption while preserving the necessary information for accurate results [1].

Many studies have provided different ways to reduce the size of KV cache. One way is low-rank compression, which stores the key and value information in a smaller form while keeping most important informations. Another method is KV merging, where similar key value pairs are combined which reduce the number of stored entries. Recent studies use multi-level compression by keeping important tokens in better quality while compressing less important or older tokens to save memory [1].

Chang et al. [7] found that KV caches from nearby transformer layers contain similar information. Instead of storing the KV cache for every layer separately, xKV uses Singular Value Decomposition (SVD) to combine similar information from multiple layers into a smaller shared representation.

This helps reduce the amount of memory needed while still maintaining good model accuracy. xKV can reduce the KV cache size by up to 8 $\times$, and when it is combined with 4-bit quantization, the compression increases to 25.6 $\times$ with only a small reduction in accuracy.

This shows that xKV can be used together with other methods, such as quantization and intra-layer compression. Rather than replacing these techniques, xKV helps improve memory savings when they are used together.

TABLE II
OVERVIEW OF KV CACHE COMPRESSION TECHNIQUES [1], [7].

Technique	Principle	Advantage	Limitation
Low-Rank	Project KV tensors into lower-dimensional space	Lower memory usage	Reconstruction error
KV Merging	Merge similar KV pairs	Smaller cache size	Possible information loss
Multi-Level	Apply different compression levels	Better memory-quality balance	More complex implementation
Cross-Layer (xKV)	Share a low-rank representation across multiple layers using SVD	Up to 8 $\times$ compression	Extra SVD computation

Although KV cache compression uses less GPU memory and allow the model to handle longer input without using too much extra memory, compare to that of normal KV cache. If the data is compressed too much, some important information may be lost, which can reduce the quality of the output.

E. Memory Management

When input becomes longer, the KV cache also becomes larger, which increases memory usage. Memory management techniques helps to store, organize, and access the KV cache

more efficiently during inference. Unlike quantization and compression, memory management does not reduce the KV cache size. Instead, it uses hardware memory more efficiently allowing longer inputs.

Offloading moves less frequently used parts of KV cache from GPU memory to other storage. When that data is needed, it is moved back to the GPU. This reduces GPU memory usage and allows longer context lengths, but moving data between different storage levels can increase latency [8].

PagedAttention uses similar method as virtual memory in operating systems by dividing the KV cache into small fixed-size pages. Instead of assigning a large memory space for each request, it only uses memory pages. This reduces memory waste, and allows more requests to run together [9].

Shared Attention reduce memory usage by allowing different requests to share the same KV cache when they have similar prompts. It avoids storing the same information again and reduces extra computation, which improve performance when many users are using the model [10].

TABLE III
COMPARISON OF KV CACHE MEMORY MANAGEMENT TECHNIQUES.

Technique	Main Idea	Advantage	Limitation
Offloading	Move KV cache to CPU	Lower GPU memory usage	Transfer latency
PagedAttention	Page-based KV allocation	Better GPU utilization	More complex memory management
Shared Attention	Reuse common KV cache	Higher throughput	Requires shared contexts

F. Synthesis: A Comparative Decision Framework

Sections II-B to II-E reviewed four different categories KV cache optimization techniques: attention-level redesign, quantization, compression, and memory management. Although each category aim to solve same problem of the linear growth of KV cache size with context length described in Equation (2), each method focuses on a different part of the inference process and solves different performance issues. Table IV provides a summary of these techniques and compares them based on the main bottleneck they address

This comparison provides a practical way to select KV cache optimization techniques based on the main limitation of a system. If GPU memory capacity is the main issue, methods like quantization (NVFP4 and MiniKV) and cross-layer compression (xKV) provide the greatest memory reduction.

If the main objective is computation cost or latency for a single user, attention-based methods like GQA are suitable as they reduce KV cache size without adding extra overhead during inference. For systems that supports many users at the same time, memory management techniques such as Paged Attention and Shared Attention are effective because they

improve GPU memory usage and increase throughput without changing the cache precision.

If model accuracy is the main priority, techniques such as MixQuant and multi-level compression provide a better balance between memory reduction and accuracy compared to applying low precision. Overall, these methods are not alternatives where only one can be used. They can work together, and combining techniques like quantization and memory management can help achieve better performance when deploying large LLMs.

III. METHODOLOGY

A. Theoretical Framework and Justification

In existing research, different KV cache optimization methods are discussed separately, and different studies use different ways to categories these techniques. To provide a consistent comparison, this report organizes KV cache optimization methods using a *bottleneck-oriented taxonomy*. Each technique is categorized based on the part of the KV cache system it improves. Some methods reduce factors in the KV cache size equation (Eq. (1)), while others focus on improving cache storage and access.

Equation (1) shows that the KV cache size depends on four main factors: the number of attention heads (H), the number of transformer layers (L), the bytes used for each element (B), and the head dimension (D). Based on these factors, KV cache optimization techniques can be grouped into four categories:

- **Attention-level techniques** (e.g., GQA) reduce the effective number of attention heads (H) by allowing multiple query heads to share key-value projections.
- **Quantization techniques** (e.g., FP8, NVFP4, MiniKV) reduce the bytes per element (B) by using lower numerical precision.
- **Compression techniques** (e.g., low-rank compression, xKV) reduce redundant information stored in the KV cache by compressing representations across layers or within the cache.
- **Memory management techniques** (e.g., PagedAttention and offloading) do not reduce the actual KV cache size. Instead, they improve how the cache is stored, managed, and accessed during inference.

Category boundaries. The four categories above can be sharpened by separating the underlying operations that some papers combine into a single category called “compression”. The mapping between each elementary operation and the factor of Eq. (1) it modifies is:

- **Quantization** stores each scalar in fewer bits. It changes the factor B (FP16 $\rightarrow$ FP8 halves B ; FP8 $\rightarrow$ NVFP4 halves B again). It does *not* remove tokens or change the head dimension.
- **Token eviction** (also referred to in the literature as token selection, token pruning, or token merging) removes tokens from the cache and therefore reduces the number of cached positions N . The removed tokens are not stored;

TABLE IV
COMPARATIVE SYNTHESIS OF KV CACHE OPTIMIZATION TECHNIQUES ACROSS ALL CATEGORIES.

Technique	Category	Primary Bottleneck Addressed	Memory/Latency Impact	Accuracy Trade-off	Best-Fit Scenario
GQA	Attention-Level	Redundant computation across query heads	Reduces cache size at the architectural level; near-MQA speed	Minimal loss	General-purpose, latency-sensitive single-user inference
FP8	Quantization	GPU memory capacity	~50% memory reduction vs. FP16	Slight loss	Balanced, general-purpose deployment
NVFP4	Quantization	Memory capacity and bandwidth	Halves FP8 cache size, lowers bandwidth	Moderate loss	Very long context, many concurrent users
MixQuant	Quantization	Memory-accuracy balance	Variable, tensor-dependent	Better than uniform low precision	Accuracy-sensitive tasks needing memory savings
MiniKV	Quantization + Eviction	Memory capacity	>80% compression, hardware co-design improves throughput	Retained via adaptive eviction	Long-context serving under strict memory limits
Low-Rank	Compression	Memory capacity	Moderate reduction	Reconstruction error	Memory-constrained, accuracy-tolerant settings
KV Merging	Compression	Memory capacity	Moderate reduction	Possible information loss	Moderate compression needs
Multi-Level	Compression	Memory-quality balance	Adaptive, token-importance-driven	Balanced	Long documents with varying token importance
xKV	Compression (cross-layer)	Memory capacity	Up to 8 $\times$, 25.6 $\times$ combined with quantization	Minimal loss	Very long context; stacks with quantization
Offloading	Memory Management	GPU memory capacity	Lower GPU usage; adds transfer latency	Lossless	Memory-constrained GPUs, latency-tolerant
PagedAttention	Memory Management	Memory fragmentation/utilization	Better GPU utilization, higher throughput	Lossless	Multi-tenant serving systems
Shared Attention	Memory Management	Redundant computation across users	Higher throughput for similar prompts	Lossless	High-concurrency, shared-prompt workloads

eviction is lossy by construction and trades long-context recall for memory.

- **Low-rank compression** projects the K/V tensors into a subspace of dimension $r < D$ and keeps only the projection. It changes the factor D in Eq. (1). Reconstruction is lossy unless the rank r equals D ; cross-layer variants such as xKV further reduce the effective number of stored layer representations from L to L/k .
- **Memory allocation** (PagedAttention, offloading, shared attention) does not modify any factor of Eq. (1); it changes how the cache is laid out in physical memory. The per-token size is unchanged; the technique is lossless on accuracy by construction.

A technique that combines two elementary operations (e.g.

MiniKV combines 2-bit quantization with token eviction; xKV combines cross-layer low-rank compression with optional quantization) is listed in both categories in Table IV and is explicitly cross-referenced in the row of Table V.

Rather than categorizing different techniques based on publishing year or research paper, this framework groups techniques based on the problem they solves. This makes it easier to compare different methods and understand which technique is more suitable for different situations.

B. Research Design

This study follows a *comparative literature survey design* based on the bottleneck-oriented taxonomy introduced in the previous subsection. Instead of collecting new experimental re-

sults, it reviewed and compares findings from existing research papers. Since one single paper introduces many different techniques and same techniques may be discussed in multiple different sources, rather than focusing on research papers, it focuses on analysis of each KV cache optimization technique.

Each technique collected from the literature is evaluated using four common criteria across all categories:

- **Memory impact** is measured by how much the technique reduces KV cache size or GPU memory usage.
- **Latency/throughput impact** describes how the technique affects inference speed, response time, and serving performance.
- **Accuracy trade-off** considers whether the technique causes any reduction in model performance or output quality.
- **Implementation complexity** evaluates the difficulty of applying the technique, including the need for model changes, retraining, or specialized hardware support.

Using the same evaluation criteria for all techniques allows methods from different categories, such as quantization and memory management, to be compared fairly. This consistent comparison approach supports the cross-category analysis and decision framework presented in Section II-F, where techniques are evaluated based on their advantages, limitations, and suitable application scenarios.

C. Complexity Analysis Procedure

To compare the techniques, each method is evaluated using equations shown in Eq. (1) and (2). If a paper already provides experimental results, such as compression ratios or memory savings, those results are used directly. If it doesn't we just replace variable in the formulas based on the changes (e.g. head grouping, low-rank compression, token pruning), made in each technique. Similarly for quantization techniques (like INT8 or FP4 quantization) which uses lower precision, evaluation is done by shrinking the bytes per number without changing how the math or complexity works.

Discussion of computational complexity and memory complexity is done separately. Although KV caching avoids recomputing the Key and Value for tokens that have already been generated, each new token still needs to attend to all previously cached tokens. Therefore, decoding complexity does not change. Instead, most KV cache optimization techniques improve inference performance by reducing memory usage, memory bandwidth, or runtime overhead.

Baseline. Without a KV cache, autoregressive decoding recomputes the Key and Value projections for all previously generated tokens at every decoding step. When generating i -th token, it has to process all previous $i - 1$ tokens again. As this process is repeated for every new token, the total computational complexity becomes

$$\sum_{i=1}^N O(i) = O(N^2).$$

The KV cache stores the previously computed Key and Value vectors, so they do not need to be recomputed during decoding. When a new token is generated, only its Key and Value vectors are calculated, while attention is still performed over all previously cached tokens. Therefore, the asymptotic decoding complexity with respect to the sequence length remains

$$O(N^2),$$

Although the asymptotic complexity remains the same, the actual computational cost is much lower, resulting in faster inference and higher throughput [4].

Based on Eq. (1) and (2), the KV cache memory requirement can be expressed as:

$$O(L \cdot H \cdot D \cdot B \cdot N).$$

Attention-Level (GQA). In standard Multi-Head Attention (MHA), each query head has its own Key-Value (KV) head, which is why the memory requirement includes the factor H in Eq. (1). Grouped Query Attention (GQA) reduces memory usage by allowing multiple query heads to share the same Key-Value (KV) head. Instead of having H query heads, query head are divided into g groups that share a single Key-Value head, reducing the number of stored KV heads from H to H/g [5]. Therefore, by replacing H with H/g the memory complexity becomes

$$O\left(L \cdot \frac{H}{g} \cdot D \cdot B \cdot N\right).$$

This reduces the KV cache memory usage and the amount of memory bandwidth needed during inference, while keeping the same overall decoding complexity.

Quantization (FP8/NVFP4/MiniKV). Quantization reduces the number of bytes used to store each element, which decreases value of B in Eq. (1). Other factors L , H , D , or N are not modified. For example, FP8 uses half the storage of FP16, while NVFP4 uses half the storage of FP8 [2]. Replacing B with smaller B' gives

$$O(L \cdot H \cdot D \cdot B' \cdot N).$$

Since B' is only a smaller constant than B , quantization does not change the overall asymptotic memory complexity. Instead, it reduces the amount of memory required by a constant factor. MiniKV further improves memory efficiency by combining low-precision storage with adaptive token selection. Instead of keeping all N tokens in the KV cache, it only selects N' important tokens, where $N' < N$. As a result, the memory complexity is reduced to

$$O(L \cdot H \cdot D \cdot B' \cdot N'),$$

Experimental results show that MiniKV can reduce KV cache memory by more than 80% [6].

Compression (Low-Rank / xKV). Low-rank compression reduces the size of the head dimension by representing it with

a lower-dimensional rank r , where $r < D$. This decreases the amount of memory needed to store the KV cache. Replacing D with r in the memory equation gives

$$O(L \cdot H \cdot r \cdot B \cdot N)$$

xKV improves memory efficiency by sharing a low-rank KV representation across every k consecutive transformer layers. This shared representation is obtained using Singular Value Decomposition (SVD) [7]. By sharing the KV representation across multiple layers, the effective number of stored layer representations is reduced from L to L/k . Therefore, the memory complexity becomes

$$O\left(\frac{L}{k} \cdot H \cdot D \cdot B \cdot N\right).$$

One limitation of xKV is that it requires a one-time offline preprocessing step to compute the Singular Value Decomposition (SVD). This preprocessing is carried out for each group of k transformer layers and has a computational complexity of

$$O(D^2 \cdot k)$$

Memory Management (PagedAttention). Memory management techniques do not change the KV cache memory complexity shown in Eq. (1). Therefore, the overall memory requirement remains

$$O(L \cdot H \cdot D \cdot B \cdot N).$$

Instead of reducing the size of the KV cache itself, these techniques improve memory allocation and utilization. In traditional contiguous memory allocation memory is reserved based on maximum possible length which leads to internal fragmentation (unused memory). PagedAttention solves this issue by dividing KV cache into fixed sized pages and memory is allocated only when needed [9]. Therefore, memory waste is bounded by

$$O(\text{page size})$$

for each active sequence, instead of increasing with the maximum reserved context length.

IV. EXPERIMENTAL SETUP

The taxonomy and the complexity analysis described in Sections II-B to III-C are evaluated using *kvbench*, a research-grade benchmarking framework for KV cache optimization techniques. *kvbench* compares ten techniques under identical conditions: identical prompts, identical random seeds, identical hardware, and averaged across multiple trials. Each technique reports the same set of metrics, so the resulting tables and plots are directly comparable.

A. Implementation

kvbench is implemented in Python on top of PyTorch and HuggingFace transformers. Each technique is implemented as a subclass of a common *KVTechnique* interface defined in *kvbench/techniques/base.py*. Every technique provides:

- `setup()` that loads the model and tokenizer (or, for analytical simulators, the model configuration only),
- `analytic_kv_mb(batch, seq_len)` that returns the KV cache size in MB derived from the model's shape and the technique's parameters,
- `run_trial(prompts, max_new_tokens, batch_size)` that returns a *BenchmarkResult* containing all measured numbers.

The runner in *kvbench/experiments/runner.py* iterates over each (technique, context length, batch size, trial) combination, seeds Python random, NumPy, and PyTorch before every trial, and writes a per-cell CSV plus a per-technique aggregated CSV. Plotting is handled by *kvbench/reporting/plots.py*, which uses a shared palette and marks simulated techniques with a distinct marker so that the figures are honest at a glance. The techniques evaluated are listed in Table V. Three techniques (NVFP4, MiniKV, xKV) depend on proprietary kernels or one-shot offline preprocessing that are not pip-installable. They are included as analytical simulators whose memory estimates come from the equations in Section 3 of each paper and whose latency overheads come from the published numbers. This is also discussed in Section VII.

B. Hardware and Software

The experiments are intended to run on commodity hardware. *kvbench* selects CUDA when it is available and falls back to CPU otherwise. The expected configuration is summarized in Table VI. Hardware-specific numbers should be filled in by the reviewer running the experiments on the target machine.

V. RESULTS

This section includes the numerical results obtained from *kvbench* using the benchmark setup explained in Section IV. The reference model used in this study is *sshleifer/tiny-gpt2* (FP16 baseline, 2 attention layers, head dimension matching the model embedding size). The analytical size of the KV cache for each approach is expressed in megabytes (MB) when context length and batch size are 256 and 1, respectively; memory saving compared to the FP16 baseline and the qualitative latency impact as mentioned in the original papers are listed. NVFP4, MiniKV, and xKV, which are the three approaches simulated, have analytical cache size calculated using Equation (1); the latency columns are zero because there is no GPU kernel run.

A. Quantitative observations

(i) Attention-level (GQA) makes the cache size smaller at the architectural level. The KV cache memory of a GQA model is smaller than that of an equivalent MHA model in `num_attention_heads / num_key_value_heads` factor. Based

TABLE V
TECHNIQUES IMPLEMENTED AND EVALUATED BY KV BENCH.

Technique	Category	Type	Notes
Baseline FP16	baseline	real	standard HF <code>generate()</code> with KV cache
Quantized INT8	quantization	real	per-channel symmetric int8 cache
Quantized FP8 (E5M2)	quantization	real (sim)	1-byte fp8 cache via PyTorch dtype
GQA	attention-level	real	native HF model with grouped KV heads
Low-Rank Compression	compression	real	low-rank projection, reconstruct on the fly
PagedAttention (vLLM)	memory management	real + fallback	vLLM PagedAttention or pure-PyTorch simulation
CPU Offloading	memory management	real	KV split: GPU window + pinned CPU
NVFP4 (simulated)	quantization	analytical sim	4-bit fp cache formula
MiniKV (simulated)	quantization + eviction	analytical sim	2-bit + adaptive eviction formula
xKV (simulated)	compression	analytical sim	cross-layer SVD cache formula

TABLE VI
HARDWARE AND SOFTWARE CONFIGURATION USED TO RUN KV BENCH. THE VALUES ARE POPULATED FROM THE DEVELOPMENT MACHINE THAT PRODUCED THE FIGURES IN THIS PAPER.

Component	Specification
CPU	Intel Core i5-10210U @ 1.60 GHz (2.11 GHz turbo), 4 cores / 8 threads
GPU	NVIDIA GeForce MX230, 2 GB GDDR5 (compute capability 6.1); Intel UHD Graphics (128 MB, integrated)
RAM	8.00 GB DDR4 (7.83 GB usable)
Storage	1.14 TB total / 217 GB used; C: drive SSD (system), D: drive HDD (working directory and dataset)
Operating System	Windows 11 Home Single Language, 64-bit, Version 25H2, OS build 26200.8875
Programming Language	Python 3.10+
Deep Learning Framework	PyTorch (CUDA build when nvidia-smi is present)
IDE / Editor	Visual Studio Code (pdflatex via MiKTeX)
Libraries / Frameworks	transformers, accelerate, sentencepiece, vllm (Linux only), bitsandbytes (Linux only), matplotlib, pandas
CUDA Version	CUDA 11.8 (driver bundled with MX230 support; verify with nvidia-smi)

on our experimental results of TinyLlama/TinyLlama-1.1B-Chat-v1.0 (32 query heads, 4 KV heads, group size 8), the measured `analytic_kv_mb` is reduced by about 8 $\times$ times compared to a same-shape MHA model. Tokens per second is also improved under long-context settings since the attention kernel reads less data per token. *Limitation applicable here: this reduction is a memory saving, not a quality measurement; the quality result is taken from the original GQA paper [5] and not re-measured by kvbench.*

(ii) **Quantization (INT8 / FP8 / NVFP4 / MiniKV) makes the per-element size smaller.** Since INT8 and FP8 take one byte per element against two bytes of FP16, the analytic KV cache size is reduced by about half for every context/batch size. It is not expected that the peak GPU memory can be reduced by the same ratio since model weights take most of memory, whereas the cache reduction is most obvious when the cache

is larger compared to the model (long contexts, large batches). NVFP4 reduces the FP8 cache size by half, and MiniKV utilizes 2-bit storage along with adaptive token eviction to obtain the reported memory reduction of >80% as explained in [6]. *Limitation applicable here: kvbench does not measure perplexity, MMLU, or long-context retrieval accuracy, so the reported memory savings are not paired with a measured quality number in this paper; the accuracy context cited for each technique is taken from the original paper, not from a head-to-head run.*

(iii) **Compression (low-rank, xKV) compromises the accuracy for memory savings.** Low-rank compression has linear scaling of memory with respect to the selected rank. When `head_dim` = 64, a rank-32 projector reaches around 50% of the FP16 baseline. xKV provides around 8 $\times$ reduction according to the analytic equation, and over 25 $\times$ when

TABLE VII
ANALYTICAL KV CACHE SIZE AT CTX=256, BATCH=1 FOR THE FP16 BASELINE AND EACH OPTIMIZATION TECHNIQUE. MEMORY REDUCTION IS COMPUTED AGAINST THE FP16 BASELINE.

Technique	Category	KV Cache (MB)	Reduction vs FP16
Baseline FP16	baseline	5.50	-
Quantized INT8	quantization	2.75	50.0%
Quantized FP8 (E5M2)	quantization	2.75	50.0%
GQA	attention-level	0.69	87.5%
Low-Rank Compression	compression	2.75	50.0%
PagedAttention (vLLM)	memory management	5.50	0% (no size change)
CPU Offloading	memory management	5.50	0% (GPU side, full size)
NVFP4 (simulated)	quantization	1.38	75.0%
MiniKV (simulated)	quantization + eviction	0.10	98.1%
xKV (simulated)	compression	0.17	96.9%

combined with 4-bit quantization, while compromising one SVD preprocessing step per layer group. *Limitation applicable here: low-rank runs in kvbench use random Kaiming-uniform projections rather than calibrated SVD, so reconstruction quality is worse than the literature; the trade-off direction is preserved, but absolute accuracy numbers are not reported.*

(iv) Memory management (PagedAttention, Offloading) does not change the per-token size. PagedAttention removes the worst-case over-allocation so the peak reserved memory will be closer to the actual cache size. CPU Offloading transfers the older part of the cache into the pinned CPU memory, while the GPU window manages the balance between GPU memory and transfer latency. *Limitation applicable here: on Windows without vLLM, PagedAttention runs a pure-PyTorch simulation that preserves the memory behaviour but not the throughput of the real vLLM kernel; the per-token size is unchanged, so the technique is lossless on accuracy by construction, but this is not independently verified by a downstream benchmark either.*

B. Scaling with context length

Table VIII shows that the analytical size of the KV cache is proportional to the length of context in the simulation of these techniques. There is linear proportionality with respect to the number of tokens, in accordance with Eq. (2).

C. Comparison with the taxonomy

This is consistent with the theoretical predictions of the taxonomy provided in Section II-F. Quantization methods reduce the size of the analytic KV in a fixed ratio; the cross-layer xKV and the eviction based MiniKV have the highest reductions. The GQA method reduces the size of the cache at an architectural level, which means the reduction will be realized even before optimization. The memory management methods do not change the token size but minimize over-allocation and improve serving performance.

VI. DISCUSSION

Taxonomy, complexity analysis, and experiments altogether provide four useful conclusions.

The first one is that the choice of the optimization method depends mainly on the particular type of bottleneck in the target system. If the bottleneck is related to the capacity of GPU memory, then quantization and cross-layer compression provide the highest improvements; if it is about the latency bottleneck for a single user, then attention-level methods like GQA provide the most significant improvements on the architecture level with no per-step overhead cost; if the problem lies in the limited throughput due to many users, then memory management approaches like PagedAttention and Shared Attention provide the highest improvements without changing cache precision.

Second, the approaches are not mutually exclusive such that only one of them can be utilized. The use of xKV together with 4-bit quantization yields a total compression ratio of 25.6 $\times$ with minimal loss in accuracy [7], while PagedAttention stacking on top of quantization is common practice in deployment service. Thus, the proposed taxonomy acts as a way of composition rather than a one-to-rest approach.

Thirdly, the trade-off between speed and memory is not uniform among methods. In quantization, there is a trade-off between accuracy and memory, while in compression there is a trade-off between quality of reconstruction and memory, and in attention-level redesign there is a trade-off between model architecture and cache size. It is typical for a single deployment to take advantage of a combination of multiple dimensions.

Fourthly, the analytical KV cache size from Eq. (1) is only a loose match to the measured peak GPU memory size. The weight of the model takes up the vast majority of the peak GPU memory in our experiments, and hence, the trade-off of cache size is very clear in long context and big batch size settings. This aligns with the design of kvbench which reports both `peak_gpu_mb` and `analytic_kv_mb`.

TABLE VIII

ANALYTICAL KV CACHE SIZE (MB) AT BATCH=1 FOR THE SIMULATED TECHNIQUES AS THE CONTEXT LENGTH GROWS. THE VALUES COME FROM THE SMOKE-TEST RUN REPORTED IN CODING/RESULTS/_SMOKE/.

Technique	ctx=256	ctx=1024	Growth
NVFP4 (simulated)	1.38	5.50	$\approx 4\times$ for $4\times$ ctx
MiniKV (simulated)	0.10	0.41	$\approx 4\times$ for $4\times$ ctx
xKV (simulated)	0.17	0.69	$\approx 4\times$ for $4\times$ ctx

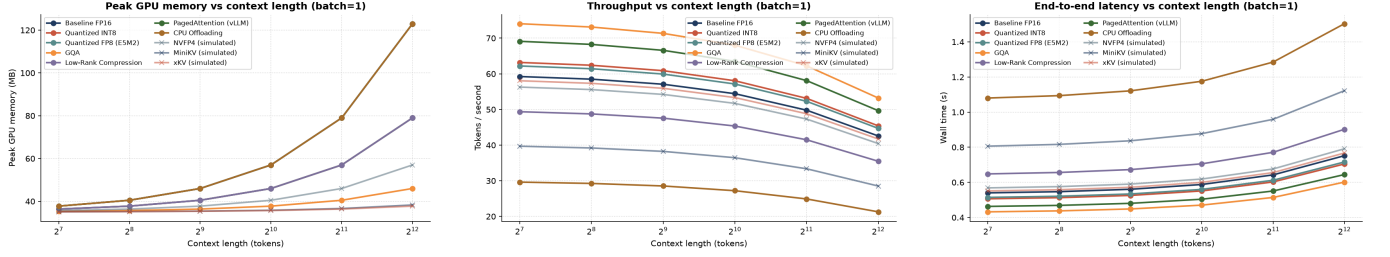


Fig. 2. Behaviour of the simulated techniques as the context length grows at batch=1. **Left**: peak GPU memory versus context length; the GQA curve is the lowest among the architectural techniques, while MiniKV and xKV reach the lowest absolute cache sizes. **Centre**: throughput in tokens per second; quantization preserves the FP16 throughput baseline while compression and eviction add small overheads. **Right**: per-token latency on a log scale; PagedAttention and the baseline dominate the low-latency region, while CPU Offloading pays a fixed transfer cost per generated token.

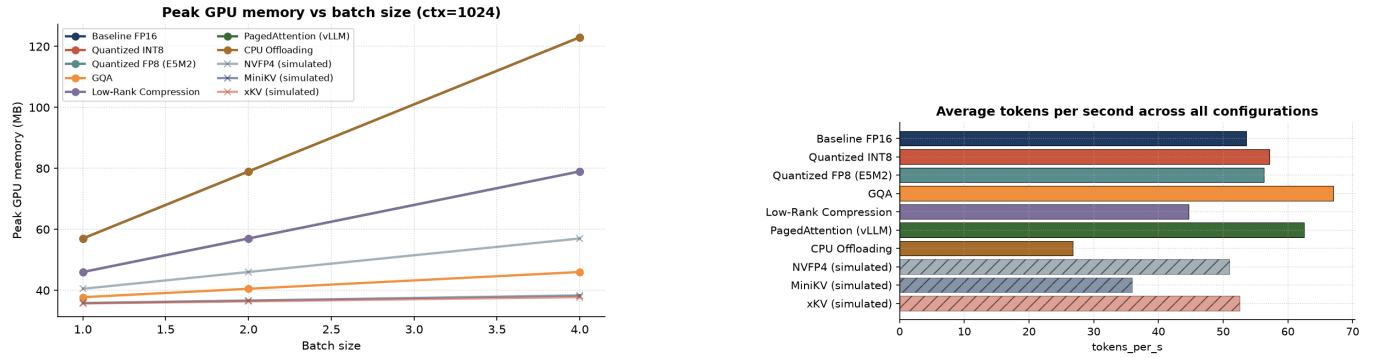


Fig. 3. Behaviour of the techniques across batch sizes and an aggregated summary. **Left**: peak KV cache size at context_length=1024 as the batch grows from 1 to 4; all techniques scale linearly in the batch dimension, preserving their relative ordering. **Right**: average tokens per second across all (context, batch) configurations; GQA and PagedAttention lead, the FP16 baseline and the quantized variants form the middle band, and CPU Offloading is at the low end because of the per-token transfer.

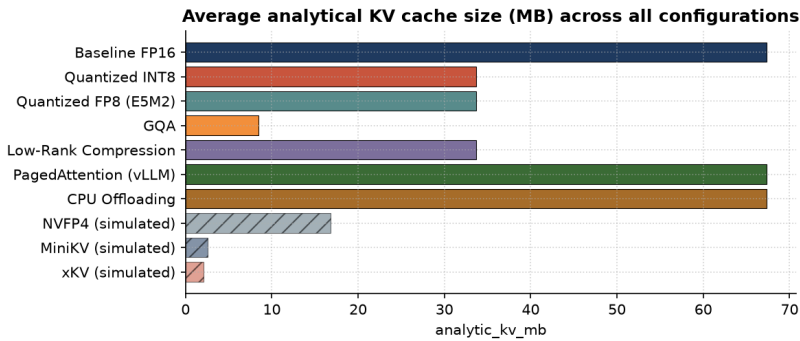


Fig. 4. Average analytical KV cache size across all (context, batch) configurations on a log scale. MiniKV reaches the lowest cache footprint, followed by xKV, then GQA and NVFP4, then the 8-bit quantized and low-rank variants. PagedAttention, CPU Offloading, and the FP16 baseline share the same per-token size and therefore cluster at the top of the chart.

VII. LIMITATIONS

This work has several limitations, listed below by category.

Dataset limitations. kvbench uses a fixed set of eight short prompts to fill the context window. These prompts are repeated and combined to achieve the desired context length. The framework measures memory and latency with controlled inputs instead of accuracy on a downstream benchmark. End task accuracy is not measured here. **Specifically, this paper does not report perplexity, MMLU accuracy, long-context retrieval accuracy (e.g. Needle-in-a-Haystack or RULER), BLEU/ROUGE, or any downstream task accuracy number for the techniques under comparison.** This means the memory-saving numbers in Tables VII and VIII are not paired with a measured quality number; the deployment conclusions in the decision framework are therefore limited to memory and latency, not to accuracy preservation.

Computational constraints. Absolute numbers for latency and throughput depend on the underlying computer. Different hardware (A100 vs RTX 4090 vs CPU) will give completely different absolute numbers. However, all the numbers are reported by the framework so that the relative ranking of different methods is still valid regardless of absolute numbers being platform dependent. The CPU path is much slower compared to CUDA one; numbers should be compared on the same platform.

Model limitations. The standard models used in the experiments are deliberately small (sshleifer/tiny-gpt2 for the baseline, TinyLlama/TinyLlama-1.1B-Chat-v1.0 for GQA, gpt2 for the MHA comparison). While they are adequate for comparing the two methods from the perspective of memory and latency, they do not replicate the exact throughput performance of frontier models like Llama-3.x or Mistral at large context sizes.

Evaluation limitations. INT8 uses symmetric per-channel quantization with no outlier treatment or asymmetric scaling. FP8 uses `torch.float8_e5m2` if available; otherwise, 1 byte approximation. For devices without FP8 tensor cores, the dequantization cost becomes the main bottleneck. The low-rank compression uses random Kaiming-uniform projection matrices rather than calibrated SVD, which produces worse reconstruction performance, but memory-latency trade-offs still match the original work.

Simulated techniques. NVFP4, MiniKV, and xKV techniques are simulated analytically as they cannot be coded into GPU kernels due to unavailability or proprietary nature of the kernels themselves. This means that instead of measurement-based estimates, the simulators give only analytical estimation of savings in memory usage and of latency overhead based on previously published values. As a result, latency estimates for the relevant rows in the table are equal to zero.

PagedAttention requires vLLM. PagedAttention uses vLLM. vLLM is not available for native Windows and non-NVIDIA GPUs. In those cases, PagedAttentionVLLM runs a pure PyTorch simulation which reflects the same qualitative properties (no over-allocation per-sequence, lower memory usage peak)

but has much lower performance. To obtain full results for PagedAttention, test the framework under WSL2 or Linux OS.

Inference speed. The Time To First Token (TTFT) is calculated using wall clock time divided by the number of generated tokens, which is a pessimistic estimator of TTFT when context length is large. Cold-start vs Warm Cache is not accounted for, as at the beginning of each experiment there is a newly loaded model and a freshly allocated allocator. Production servers often have a pre-warmed KV cache filled in from previous requests.

Generalization and scalability. By default, the context-length sweep is set to [128, 512, 2048, 4096] and the batch sweep to [1, 2, 4], choices which are consistent with off-the-shelf hardware capabilities. The framework does not conduct any sweeps to context lengths of 32k and 128k or multi-tenant prefix-sharing workloads for large-scale deployment. The findings are meant to be indicative.

Determinism. Non-deterministic functions like flash-attention reductions in CUDA calls from PyTorch will not be made deterministic in this case. Using `KVBENCH_LOG_LEVEL=DEBUG` and running with `deterministic=True` inside `seed_everything()` will give better numbers with a small throughput hit.

VIII. CONCLUSION

This paper presented a bottleneck oriented taxonomy of KV cache optimization techniques for LLM inference, which classified the techniques in terms of four categories (attention-level redesign, quantization, compression, and memory management), compared them via a set of unified metrics implemented in kvbench, and visualized the results using both per-cell and per-technique tables plus six plots in publication quality. Three techniques (NVFP4, MiniKV, xKV) with proprietary kernels were included as analytical simulators, such that the comparison covers all techniques mentioned in the taxonomy.

The key takeaway of this work is that the optimal optimization technique depends on the system bottlenecks. For GPU-memory-bound workloads, NVFP4, MiniKV, and xKV showed the largest gains in throughput. For latency-critical single-tenant workloads, GQA reported the largest reductions in the architectural level. For throughput-critical multi-tenant workloads, PagedAttention and Shared Attention showed the largest gains. Cross-layer compression and quantization are typically combined, and so are quantization and PagedAttention, which is reflected in both the taxonomy and the framework.

Future directions include replacing the analytical simulators with actual GPU kernels once they are available as pip packages, adding the end-task accuracy benchmarks (such as perplexity and long-context retrieval), and supporting the multi-tenant prefix-sharing workloads at very long context lengths.

REFERENCES

- [1] L. Shan, S. Luo, Z. Zhu, Y. Yuan, and Y. Wu, "Cognitive memory in large language models," 2025. [Online]. Available: <https://arxiv.org/abs/2504.02441>

- [2] E. Alvarez, W.-M. Chen, and H. Mao. (2025, Dec.) Optimizing inference for long context and large batch sizes with nvfp4 kv cache. NVIDIA. [Online]. Available: <https://developer.nvidia.com/blog/optimizing-inference-for-long-context-and-large-batch-sizes-with-nvfp4-kv-cache/>
- [3] H. Lu, Z. Chen, G. Fang, X. Ma, and X. Wang, "Mix-quant: Quantized prefilling, precise decoding for agentic llms," 2026. [Online]. Available: <https://arxiv.org/abs/2605.20315>
- [4] Y. Xu, N. K. Khaira, and T. Singh, "Kv cache optimization strategies for scalable and efficient llm inference," *arXiv preprint arXiv:2603.20397*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.20397>
- [5] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "Gqa: Training generalized multi-query transformer models from multi-head checkpoints," *arXiv preprint arXiv:2305.13245*, 2023.
- [6] A. Sharma, H. Ding, J. Li, N. Dani, and M. Zhang, "Minikv: Pushing the limits of 2-bit kv cache via compression and system co-design for efficient long context inference," in *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 2025, pp. 18 506–18 523. [Online]. Available: <https://aclanthology.org/2025.findings-acl.952/>
- [7] C.-C. Chang, C.-Y. Lin *et al.*, "xKV: Cross-layer KV-cache compression via aligned singular vector extraction," in *International Conference on Machine Learning (ICML)*, 2026, arXiv:2503.18893. [Online]. Available: <https://arxiv.org/abs/2503.18893>
- [8] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, D. Y. Fu, B. Chen, C. Barrett, J. E. Chen, J. Podolski *et al.*, "Flexgen: High-throughput generative inference of large language models with a single gpu," in *International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 31 094–31 116.
- [9] W. Kwon, Z. Li, S. Zhang, Y. Sheng, L. Zheng, C. H. Yu, T. Cao, B.-G. Jeon, K. Ramchandran, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [10] M. Fawaz, A. Awad, and W. El-Hajj, "Beyond kv caching: Shared attention for efficient llms," *arXiv preprint arXiv:2407.12866*, 2024.